# Netflix Stock Price Prediction

## Problem Statement

**Background:** Netflix is a popular streaming service that provides a wide range of movies, TV shows, and original content to its subscribers. As a publicly traded company, the stock price of Netflix is subject to fluctuations in the market. Investors and traders are always interested in predicting the future stock prices to make informed decisions regarding buying or selling Netflix stocks.

**Dataset Information:** The dataset used in this project is the Netflix stock price dataset. It contains historical stock price data for Netflix, including the opening price, high price, low price, volume, and closing price for each trading day. The dataset provides a time series of Netflix stock prices, allowing us to analyze patterns and trends in the stock market.

**Problem Statement:** The goal of this project is to develop a machine learning model that can predict the future stock prices of Netflix based on historical data. By accurately predicting the stock prices, investors and traders can make more informed decisions about buying or selling Netflix stocks, potentially maximizing their profits.

**Approach:** To tackle this problem, we will use the linear regression algorithm, a popular machine learning algorithm for regression tasks. Linear regression models the relationship between the independent variables (opening price, high price, low price, volume) and the dependent variable (closing price) using a linear equation. The model will learn from the historical data and then make predictions on unseen data.

**Workflow:**

- Load the Netflix stock price dataset from a CSV file.
- Perform data preparation steps, including checking for missing values, exploring the dataset's shape and information, and analyzing descriptive statistics.

- Split the dataset into training and testing sets. The training set will be used to train the linear regression model, and the testing set will be used to evaluate the model's performance.
- Prepare the input features (opening price, high price, low price, volume) and target variable (closing price) for training and testing.
- Train the linear regression model on the training data using the LinearRegression() class from scikit-learn.
- Make predictions on the testing data using the trained model.
- Evaluate the performance of the model by calculating various metrics such as mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), mean absolute percentage error (MAPE), and R-squared score.
- Visualize the actual closing prices versus the predicted closing prices using scatter plots and line plots.
- Save the predicted closing prices along with the corresponding dates to a CSV file for further analysis and reference.

By following this approach, we can build a linear regression model that can predict the future stock prices of Netflix based on historical data. This prediction can assist investors and traders in making more informed decisions regarding Netflix stock investments.

# Framework

**Importing the Required Libraries:** Begin by importing the necessary libraries for data manipulation, visualization, and machine learning. Import numpy, pandas, matplotlib.pyplot, train_test_split from sklearn.model_selection, LinearRegression from sklearn.linear_model, and various metrics from sklearn.metrics. Additionally, import the os library for file handling and warnings library to ignore any warnings.

**Loading the Dataset:** Load the Netflix stock price dataset from a CSV file. Use the pd.read_csv() function to read the file and store it in a pandas DataFrame named df.

**Data Exploration and Preparation:** Perform data exploration to gain insights into the dataset. Check for missing values using the isnull().sum() function and observe the shape and information of the dataset using the shape and info() functions, respectively. Analyze descriptive statistics using the describe().T function.

**Data Splitting:** Split the dataset into training and testing sets using the train_test_split() function from the sklearn.model_selection module. Set the test_size parameter to define the proportion of data to be used for testing (e.g., 0.2 for 20% test data).

**Data Preparation for Model Training:** Prepare the input features and target variable for training and testing. Extract the required columns ('Open', 'High', 'Low', 'Volume') from the training and testing sets and convert them into NumPy arrays. Assign the 'Close' column as the target variable.

**Model Training:** Create an instance of the LinearRegression() class and train the linear regression model using the training data. Fit the model to the training features (x_train) and target variable (y_train) using the fit() function.

**Making Predictions:** Use the trained model to make predictions on the testing data. Apply the predict() function on the testing features (x_test) to obtain the predicted values of the target variable (y_pred).

**Model Evaluation:** Evaluate the performance of the model by calculating various metrics. Use the appropriate functions (mean_squared_error(), np.sqrt(), mean_absolute_error(), mean_absolute_percentage_error(), r2_score()) to calculate metrics like mean squared error (MSE), root mean squared error (RMSE), mean absolute

error (MAE), mean absolute percentage error (MAPE), and R-squared score. Print the results.

**Model Visualization:** Visualize the actual closing prices versus the predicted closing prices using scatter plots and line plots. Create a function style() to customize the plot's aesthetics. Plot the closing prices over time using plt.plot() and add legends, titles, and color customization.

**Saving the Predictions:** Convert the 'Date' column in the testing dataset to the datetime format. Create a new DataFrame output containing the date, actual closing price, and predicted closing price. Save this DataFrame as a CSV file named 'Close_Prediction.csv' using the to_csv() function. Print a success message once the file is saved.

# Code Explanation

**Importing Libraries:** In this section, we import the necessary libraries for our project. These libraries provide various functionalities for data manipulation, visualization, and machine learning. Some of the libraries used in the code are numpy, pandas, matplotlib.pyplot, and scikit-learn (sklearn). We import these libraries to leverage their functions and classes for our tasks.

**Data Loading:** This section focuses on loading the dataset into our program. We use the pandas library's read_csv() function to read the data from a CSV file. The file path is provided as an argument to the function. The dataset is then stored in a pandas DataFrame, which is a tabular data structure that allows us to perform operations on the data easily.

**Data Preparation:** After loading the dataset, we perform some initial data preparation steps. We check for any missing values in the dataset using the isnull().sum() function, which returns the count of missing values in each column. This helps us ensure the quality of our data.

**Data Splitting:** To evaluate the performance of our model, we need to split the dataset into training and testing sets. The train_test_split() function from the sklearn.model_selection module is used for this purpose. It randomly divides the data into two sets based on the specified proportion (in this case, 80% training and 20% testing). This step allows us to train the model on one set and evaluate its performance on unseen data from the other set.

**Data Preparation for Model Training:** After splitting the data, we prepare the input features and target variable for training the model. We extract the required columns ('Open', 'High', 'Low', 'Volume') from the training and testing sets and convert them into NumPy arrays. This conversion is necessary to feed the data into the machine learning model. Additionally, we assign the 'Close' column as the target variable, which we aim to predict.

**Model Training:** This section involves training our machine learning model. We use the LinearRegression class from the sklearn.linear_model module. First, we create an instance of the LinearRegression class called model_lnr. Then, we use the fit() method of this instance to train the model on the training features (x_train) and target variable

(y_train). This process involves finding the best fit line that minimizes the difference between the predicted values and the actual target values.

**Making Predictions:** Once the model is trained, we can use it to make predictions on the testing data. We apply the predict() method on the testing features (x_test) to obtain the predicted values of the target variable (y_pred). This step allows us to compare the model's predictions with the actual values to evaluate its performance.

**Model Evaluation:** In this section, we evaluate the performance of our model by calculating various metrics. These metrics help us understand how well the model is performing in terms of prediction accuracy. The code calculates metrics such as mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), mean absolute percentage error (MAPE), and R-squared score. These metrics provide insights into the model's predictive capability and how well it fits the data.

**Model Visualization:** To visualize the results, we use matplotlib.pyplot to create plots. The code includes a function called style() that customizes the plot's aesthetics, such as the background color, tick colors, and spine colors. We plot the closing stock prices over time using plt.plot() and add legends, titles, and color customization to make the plot visually appealing and informative.

**Saving the Predictions:** Finally, we save the predictions made by our model as a CSV file. We create a new DataFrame containing the date, actual closing price, and predicted closing price. This DataFrame is then saved as a CSV file named 'Close_Prediction.csv' using the to_csv() function. This allows us to further analyze the predictions or use them for other purposes.

# Future Work

Predicting stock prices is a challenging task that involves various factors such as market trends, investor sentiment, and company performance. While the current project focuses on using linear regression to predict Netflix stock prices, there are several avenues for future work and improvements. Here is a detailed step-by-step guide on implementing future work for predicting Netflix stock prices:

**1. Feature Engineering and Selection:**

- Explore additional features: Consider incorporating additional features that might influence stock prices, such as macroeconomic indicators (e.g., GDP growth, interest rates), industry-specific data, or news sentiment analysis.
- Perform feature selection: Analyze the importance and relevance of existing features and potentially remove less significant ones. Techniques like correlation analysis, feature importance ranking, or dimensionality reduction methods like principal component analysis (PCA) can be employed.

**2. Advanced Machine Learning Algorithms:**

- Explore advanced algorithms: Besides linear regression, try implementing more sophisticated machine learning algorithms such as decision trees, random forests, support vector machines (SVM), or neural networks. These algorithms may capture complex relationships in the data and potentially improve prediction accuracy.
- Model ensemble techniques: Combine predictions from multiple models using ensemble techniques like bagging, boosting, or stacking. This approach can help mitigate the limitations of individual models and enhance overall performance.

**3. Time Series Analysis and Modeling:**

- Consider time series techniques: As stock prices are time-dependent data, apply time series analysis methods like autoregressive integrated moving average (ARIMA), exponential smoothing (e.g., Holt-Winters), or more advanced models like long short-term memory (LSTM) networks, capable of capturing temporal dependencies in the data.

- Incorporate lagged variables: Introduce lagged variables, representing past values of the target variable or other relevant features, to account for historical patterns and trends that can impact future prices.

**4. Hyperparameter Tuning and Model Optimization:**

- Optimize model hyperparameters: Fine-tune the hyperparameters of the selected algorithms to find the optimal configuration that yields the best performance. Utilize techniques like grid search, random search, or Bayesian optimization to automate the process and avoid manual parameter tuning.
- Cross-validation: Implement robust cross-validation techniques such as k-fold cross-validation to evaluate model performance across different subsets of the data. This helps in assessing model generalization and identifying potential overfitting or underfitting issues.

**5. Incorporating External Data Sources:**

- Integrate external data: Include data from relevant external sources such as financial news sentiment, social media trends, or market indices to capture additional market dynamics and sentiment analysis. These data sources can provide valuable insights into market behavior and potentially enhance prediction accuracy.

**6. Evaluation and Metrics:**

- Refine evaluation metrics: Explore additional evaluation metrics suitable for stock price prediction, such as mean absolute percentage error (MAPE), directional accuracy, or profit-based metrics that consider trading strategies based on predicted prices.
- Comparative analysis: Compare the performance of different models and techniques using various metrics to identify the most effective approach.

**7. Real-time Prediction and Deployment:**

- Implement real-time prediction: Develop a system that can fetch real-time data and provide up-to-date predictions of Netflix stock prices. This involves automating data retrieval, model updating, and prediction generation.

- Deployment options: Explore deployment options, such as creating a web-based interface or API to access the predictions, enabling users to interact with the model and make informed decisions.

**Step-by-Step Guide to Implement Future Work:**

1. Perform exploratory data analysis on the existing dataset and identify potential new features that may influence stock prices.
2. Preprocess and clean the data, handling missing values and outliers as necessary.
3. Implement feature engineering techniques to create additional relevant features.
4. Split the dataset into training and testing sets, considering an appropriate time frame to capture temporal dependencies.
5. Explore advanced machine learning algorithms like decision trees, random forests, or neural networks. Implement and evaluate their performance using appropriate evaluation metrics.
6. Apply time series analysis techniques like ARIMA, exponential smoothing, or LSTM networks to model temporal dependencies in the data.
7. Optimize the models by tuning hyperparameters and utilizing cross-validation techniques.
8. Incorporate external data sources such as financial news sentiment or market indices to enhance prediction accuracy.
9. Evaluate the models using a comprehensive set of metrics and compare their performance.
10. Develop a real-time prediction system that fetches and updates data automatically, providing up-to-date predictions.
11. Deploy the model through a web-based interface or API to enable users to access and utilize the predictions.

By following this step-by-step guide and incorporating the suggested future work, you can further enhance the accuracy and robustness of the Netflix stock price prediction model. Remember to iterate and experiment with different approaches to find the best combination of techniques that suit the specific requirements of the project.

# Concept Explanation

Imagine you're on a quest to predict the future, specifically the stock prices of Netflix. How can you do that? Well, one approach is to use a magical algorithm called Linear Regression. It's like having a crystal ball, but instead of magic, it uses math!

Linear Regression is a popular algorithm that tries to find a straight line that best fits the relationship between two or more variables. In our case, we want to find a line that can predict Netflix's stock price based on certain factors like the opening price, high price, low price, and trading volume.

Let's break it down step by step:

**Gather the Data:** First, we need historical data of Netflix stock prices along with the corresponding factors like opening price, high price, low price, and volume. Think of it as collecting clues about the past to predict the future.

**Preparing the Data:** We clean and organize the data, ensuring there are no missing values or discrepancies. We want our data to be squeaky clean, just like a freshly laundered Netflix t-shirt.

**Split the Data:** Now we divide our data into two sets: a training set and a testing set. It's like splitting your Netflix binge-watching time between watching new episodes and taking breaks to test your knowledge.

**Training Time:** Here comes the fun part! We use the training set to teach our algorithm the relationship between the factors (opening price, high price, low price, and volume) and the target variable (Netflix's closing price). The algorithm learns from this training data, just like you learn from watching Netflix's previous episodes.

**Predicting the Future:** Once our algorithm has learned the patterns, it's time to put it to the test! We feed it the factors for the testing set, and it predicts the closing prices for those days. It's like asking our algorithm, "Hey, what do you think the stock price will be based on these factors?" It's a bit like predicting the cliffhanger of the next episode!

**Evaluation Time:** We assess how well our algorithm performed. We calculate metrics like Mean Squared Error, Root Mean Squared Error, Mean Absolute Error, and R-squared score to measure its accuracy. It's like giving our algorithm a report card to see if it passed the prediction test with flying colors.

**Visualization:** To make things more exciting, we create visualizations. We plot the actual closing prices and the predicted closing prices on a graph. It's like comparing the script of the episode with what actually happened. Did our algorithm get it right?

**Saving the Day:** We save the predictions as a CSV file, just like bookmarking your favorite Netflix series. You can refer to it later to see how well the algorithm performed.

That's the magic of Linear Regression! It helps us uncover the hidden patterns and relationships in the data to make predictions. But remember, it's not foolproof. The stock market can be as unpredictable as a Netflix plot twist. So use these predictions wisely and always stay curious to explore and improve your algorithms further.

# Exercise Questions

**Question: Why is data splitting important in machine learning projects like stock price prediction?**

**Answer:** Data splitting is crucial because it allows us to evaluate the performance of our machine learning model on unseen data. By dividing our dataset into training and testing sets, we can train the model on the training data and then assess its accuracy and generalization ability on the testing data. This helps us understand how well the model will perform on new, unseen data in the real world.

**Question: What are some evaluation metrics used to assess the performance of a stock price prediction model?**

**Answer:** There are several evaluation metrics we can use to measure the performance of a stock price prediction model. Some commonly used metrics include:

- Mean Squared Error (MSE): It measures the average squared difference between the predicted and actual values, providing a measure of how well the model's predictions match the actual values.
- Root Mean Squared Error (RMSE): It is the square root of the MSE, giving us an estimate of the average prediction error in the original units of the target variable.
- Mean Absolute Error (MAE): It calculates the average absolute difference between the predicted and actual values, providing a measure of the model's average prediction error.
- Mean Absolute Percentage Error (MAPE): It measures the percentage difference between the predicted and actual values, giving us an indication of the average relative error of the model's predictions.
- R-squared (R2) Score: It represents the proportion of the variance in the target variable that can be explained by the model. A higher R2 score indicates a better fit of the model to the data.

**Question: How does Linear Regression work in predicting stock prices?**

**Answer:** Linear Regression is a predictive modeling algorithm that seeks to find the best-fit line to represent the relationship between the independent variables (such as opening price, high price, low price, and volume) and the dependent variable (closing

price). In the context of stock price prediction, Linear Regression uses historical data to identify patterns and trends in the independent variables and their impact on the closing price. By fitting a linear equation to the data, the algorithm can make predictions about future stock prices based on the given factors.

**Question: Can Linear Regression handle non-linear relationships between the independent and dependent variables?**

**Answer:** No, Linear Regression assumes a linear relationship between the independent and dependent variables. If the relationship is non-linear, Linear Regression may not capture the underlying patterns accurately. In such cases, more advanced techniques like polynomial regression or other non-linear regression algorithms may be more appropriate.

**Question: What are some limitations or challenges of using Linear Regression for stock price prediction?**

**Answer:** While Linear Regression can be a useful tool for stock price prediction, it does have limitations and challenges:

- Linear Regression assumes a linear relationship between the factors and the stock price, which may not always hold true in complex financial markets.
- It may not capture non-linear trends and patterns in the data, leading to inaccurate predictions.
- The model's performance heavily depends on the quality and relevance of the chosen factors. Selecting the right features is crucial for accurate predictions.
- Linear Regression is sensitive to outliers and noise in the data. Outliers can significantly affect the estimated coefficients and, subsequently, the predictions.
- The stock market is influenced by various external factors and events, such as economic indicators, news, and geopolitical events, which may not be captured solely by the given factors used in Linear Regression.
- The predictions are based on historical data, and the future stock market behavior may deviate from past patterns, making it challenging to capture unexpected market changes.